

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ федерального государственного бюджетного
образовательного учреждения высшего образования
«РОССИЙСКИЙ ЭКОНОМИЧЕСКИЙ УНИВЕРСИТЕТ ИМЕНИ
Г.В.ПЛЕХАНОВА»
Техникум Пермского института (филиала)

Учебная практика
по дисциплине «Осуществление интеграции программных модулей»
Тема: «Разработка приложения TaskManager»
Студента группы ИПо-32
Новикова Александра Олеговна

2026 г.

Оглавление

Описание программы.....	3
Проектирование системы TaskManager.....	3
Создание Проекта.....	4
Создание Библиотеки TaskManagerData.....	6
Создание модели данных.....	6
Создание сервиса для управления данными.....	8
Создание сервиса для загрузки/выгрузки данных в виде файла.....	12
Каталог библиотеки классов.....	14
Создание Wpf-приложения.....	15
Создание представлений и поведения.....	15
Структура проекта.....	17
Внешний вид.....	17
Создание модульных тестов для классов-сервисов.....	18
Тестирование TaskService.....	18
Тестирование FileService.....	21
Результаты тестирования.....	25

Описание программы

Проектирование системы TaskManager

Система состоит из двух основных модулей.

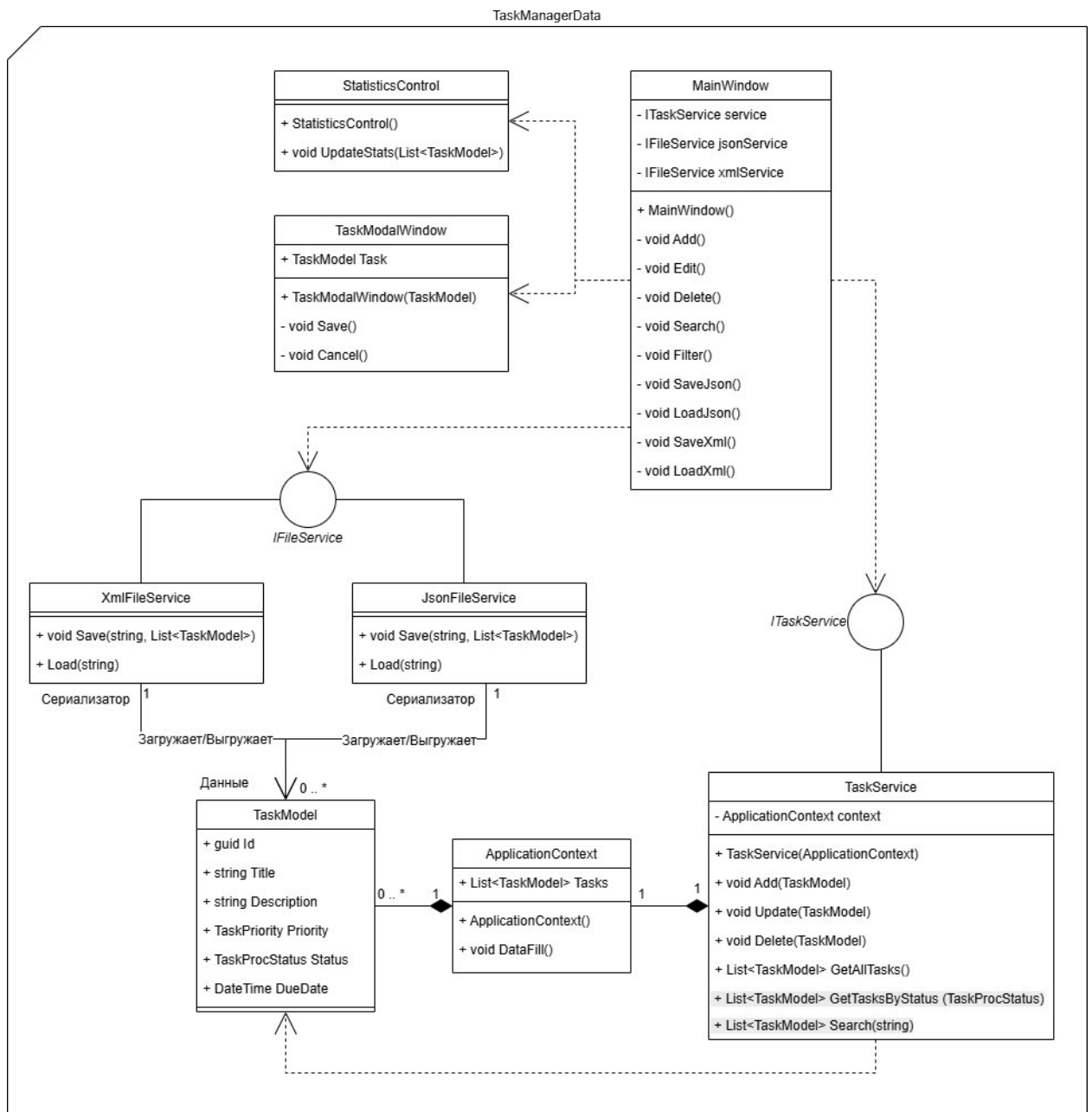
Модуль TaskManagerData реализует основную логику работы программы и манипуляции с данными.

Модель данных реализуется в классе *TaskModel*. В нем перечислены основные характеристики объекта Задача: идентификатор, Название, Описание, Приоритет, Статус, Дата срока исполнения. *ApplicationContext* представляет контейнер для коллекции объектов-задач. Манипуляция объектами-задачами реализуется в классе *TaskService*. Логика работы с задачами в этом классе описана через методы по созданию, изменению, удалению отдельного объекта-задачи, а также чтению разных списков задач: полного списка, списка по найденному заголовку/описанию, списка по статусу задач.

В классах *XmlFileService*, *JsonFileService* реализуется сериализация/десериализация и выгрузка/загрузка задач в файлах формата json, xml.

Интерфейсы *IFileService* и *ITaskService* представляют контракты для классов-сервисов. Они перечисляют методы, которые должны быть реализованы в сервисах и через которые будет происходить дальнейшее взаимодействие с другой частью программы.

Графический интерфейс представлен классами *MainWindow* *StatisticsControl* *TaskModalWindow*. *MainWindow* – основной класс, который использует сервисы, а также пользовательский элемент *StatisticsControl* для вывода статистики и Окно создания/изменения задачи *TaskModalWindow*.



Создание Проекта

Для разработки программы используется IDE Visual Studio, платформа .Net 8.0, язык программирования C#, графическая подсистема WPF для создания пользовательского интерфейса.

Первым делом создаем проект TaskManager

Приложение WPF (Майкрософт)

C#

Windows

Рабочий стол

Имя проекта

TaskManager

Расположение

C:\Learn

Имя решения ⓘ

TaskManager

☐ Поместить решение и проект в одном каталоге

Проект будет создан в "C:\Learn\TaskManager\TaskManager\"

Приложение WPF (Майкрософт)

C#

Windows

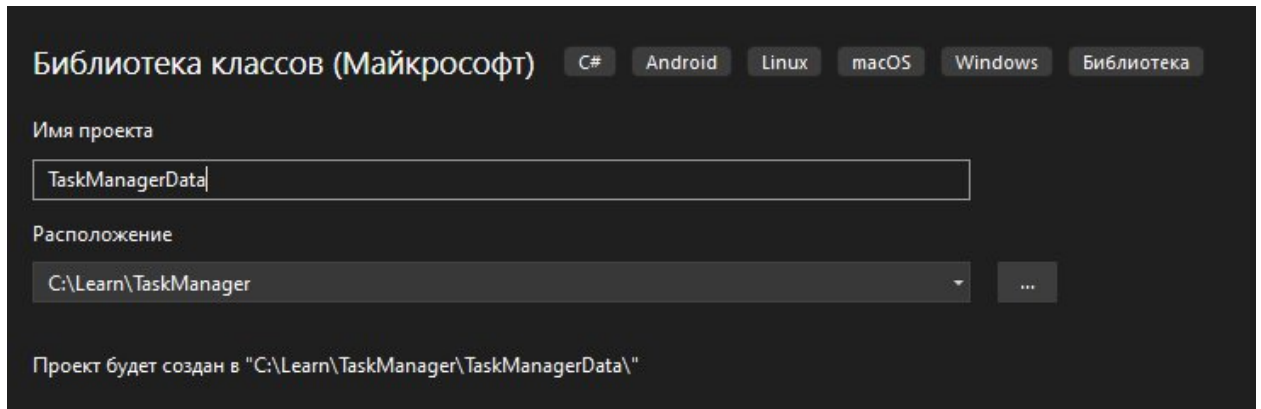
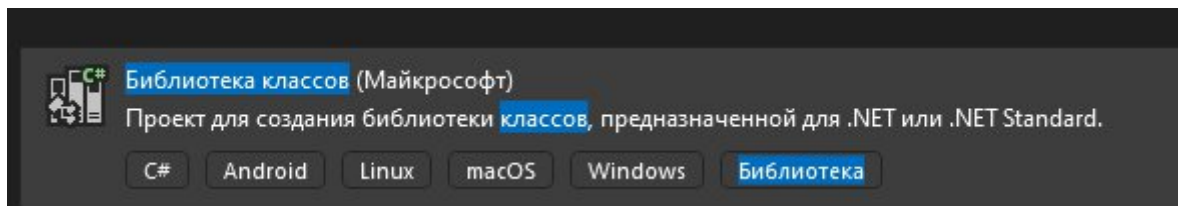
Рабочий стол

Платформа ⓘ

.NET 8.0 (долгосрочная поддержка)

Создание Библиотеки TaskManagerData

В проекте TaskManager создаем проект библиотеки классов TaskManagerData и добавляем в основной проект зависимость на библиотеку



Диспетчер ссылок - TaskManager

Проекты		
Решение	Имя	Путь
Общие проекты	☒ TaskManagerData	C:\Learn
	TaskManagerDataTests	C:\Learn

Создание модели данных

Создаем класс модели данных «Задача» и перечисления для полей статуса и приоритетности задачи

```

namespace TaskManagerData.Models
{
    public class TaskModel
    {
        public Guid Id { get; set; } = Guid.NewGuid();

        public string Title { get; set; } = string.Empty;

        public string Description { get; set; } = string.Empty;

        public TaskPriority Priority { get; set; }

        public DateTime DueDate { get; set; }

        public TaskProcStatus Status { get; set; } = TaskProcStatus.New;
    }
}

```

```

namespace TaskManagerData.Enums
{
    public enum TaskPriority
    {
        Low,
        Medium,
        High
    }
}

```

```

namespace TaskManagerData.Enums
{
    public enum TaskProcStatus
    {
        New,
        InProgress,
        Completed
    }
}

```

Создаем контекст приложения, в котором будет храниться список задач, через контекст будет реализовано последующее управление задачами

```

namespace TaskManagerData.Context
{
    public class ApplicationContext
    {
        public List<TaskModel> Tasks;

        public ApplicationContext()
        {
            Tasks = new List<TaskModel>();

            DataFill();
        }

        public void DataFill()
        {
        }
    }
}

```

Создание сервиса для управления данными

Далее пишем интерфейс для сервиса, управляющего задачами. В нем фиксируются методы, которые должны быть в самом сервисе, и которые будут использованы в WPF-модуле.


```
namespace TaskManagerData.Services.Interfaces
{
    public interface ITaskService
    {
        bool Add(TaskModel task);

        bool Update(TaskModel task);

        bool Delete(TaskModel task);

        List<TaskModel> GetAllTasks();

        List<TaskModel> GetTasksByStatus(TaskProcStatus status);

        List<TaskModel> Search(string text);
    }
}
```

После этого пишем сам сервис работы с задачами и реализацию всех методов

```
namespace TaskManagerData.Services
{
    public class TaskService : ITaskService
    {
        private readonly ApplicationContext _context;

        public TaskService(ApplicationContext context)
        {
            _context = context;
        }

        public bool Add(TaskModel task)
        {
            bool result = false;
            if (task != null)
            {
                if (!String.IsNullOrEmpty(task.Title)
                    && task.DueDate > DateTime.Now)
                {
                    _context.Tasks.Add(task);
                    result = true;
                }
            }
            return result;
        }
    }
}
```

```

public List<TaskModel> GetAllTasks()
{
    return _context.Tasks;
}

public List<TaskModel> GetTasksByStatus(TaskProcStatus status)
{
    return _context.Tasks
        .Where(t => t.Status == status)
        .ToList();
}

public List<TaskModel> Search(string text)
{
    return _context.Tasks
        .Where(t =>
            t.Title.Contains(text,
                StringComparison.OrdinalIgnoreCase)
            ||
            t.Description.Contains(text,
                StringComparison.OrdinalIgnoreCase))
        .ToList();
}

```

```

public bool Update(TaskModel task)
{
    bool result = false;

    if (task != null)
    {
        if (!String.IsNullOrEmpty(task.Title))
        {
            var target = _context.Tasks.FirstOrDefault(x => x.Id == task.Id);
            if (target != null)
            {
                if (target.DueDate <= task.DueDate)
                {
                    int id = _context.Tasks.IndexOf(target);
                    if (id != -1) _context.Tasks[id] = task;

                    result = true;
                }
            }
        }
    }
    return result;
}

```

```

public bool Delete(TaskModel task)
{
    bool result = false;
    if (task != null)
    {
        if (_context.Tasks.Contains(task))
        {
            _context.Tasks.Remove(task);
            result = true;
        }
    }
    return result;
}

```

Создание сервиса для загрузки/выгрузки данных в виде файла

Далее пишем интерфейс с двумя методами, для загрузки выгрузки файлов, которые должны быть реализованы в сервисах для выгрузки, и к которым будет доступ в WPF-модуле.

```

namespace TaskManagerData.Services.Interfaces
{
    public interface IFileService
    {
        void Save(string path, List<TaskModel> tasks);

        List<TaskModel> Load(string path);
    }
}

```

Далее пишем классы выгрузки в виде Json Xml

```

namespace TaskManagerData.Services
{
    public class JsonFileService : IFileService
    {
        public void Save(string path, List<TaskModel> tasks)
        {
            string json = JsonSerializer.Serialize(
                tasks,
                new JsonSerializerOptions
                {
                    WriteIndented = true
                });

            File.WriteAllText(path, json);
        }

        public List<TaskModel> Load(string path)
        {
            if (!File.Exists(path))
            {
                throw new FileNotFoundException("Файл не найден.", path);
            }

            string json = File.ReadAllText(path);

            return JsonSerializer.Deserialize<List<TaskModel>>(json)
                ?? new List<TaskModel>();
        }
    }
}

```



```

namespace TaskManagerData.Services
{
    public class XmlFileService : IFileService
    {
        public void Save(string path, List<TaskModel> tasks)
        {
            XmlSerializer serializer =
                new XmlSerializer(typeof(List<TaskModel>));

            using FileStream stream = new FileStream(
                path,
                FileMode.Create);

            serializer.Serialize(stream, tasks);
        }

        public List<TaskModel> Load(string path)
        {
            if (!File.Exists(path))
            {
                throw new FileNotFoundException("Файл не найден.", path);
            }

            XmlSerializer serializer =
                new XmlSerializer(typeof(List<TaskModel>));

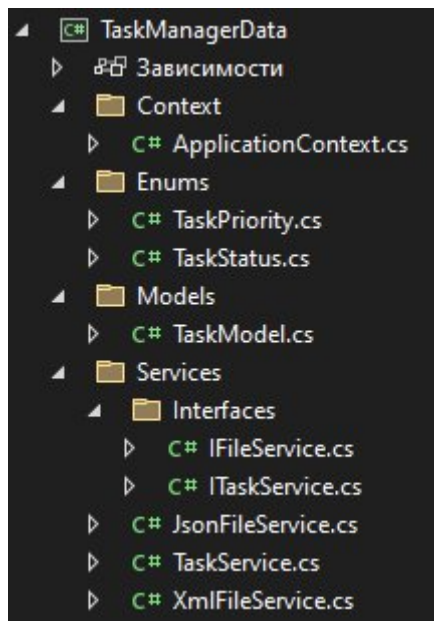
            using FileStream stream = new FileStream(
                path,
                FileMode.Open);

            return (List<TaskModel>)serializer.Deserialize(stream)!;
        }
    }
}

```

Каталог библиотеки классов

Классы библиотеки классов расположены в проекте следующим образом



Создание Wpf-приложения

Создаем общий словарь стилей для элементов графического интерфейса

DictionaryBasic.xaml

```
<ResourceDictionary xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
                    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml">
    <Style TargetType="Button">
        <Setter Property="Margin" Value="5"/>
        <Setter Property="Padding" Value="10,6"/>
        <Setter Property="FontSize" Value="14"/>
        <Setter Property="MinWidth" Value="110"/>
    </Style>

    <Style TargetType="TextBox">
        <Setter Property="Margin" Value="5"/>
        <Setter Property="Padding" Value="5"/>
    </Style>

    <Style TargetType="ComboBox">
        <Setter Property="Margin" Value="5"/>
    </Style>

    <Style TargetType="Label">
        <Setter Property="Margin" Value="5"/>
        <Setter Property="FontWeight" Value="SemiBold"/>
    </Style>
</ResourceDictionary>
```

Создание представлений и поведения

Создаем представление и поведение модального окна для создания/изменения объекта задание

TaskModalWindow.xaml.cs

```
public partial class TaskModalWindow : Window
{
    public TaskModel Task { get; private set; }

    public TaskModalWindow(TaskModel task = null)
    {
        InitializeComponent();

        cbPriority.ItemsSource = Enum.GetValues(typeof(TaskPriority));
        cbStatus.ItemsSource = Enum.GetValues(typeof(TaskProcStatus));

        if (task == null)
        {
            Task = new TaskModel();
            dpDate.SelectedDate = DateTime.Now.AddDays(1);
        }
        else
        {
            Task = task;

            tbTitle.Text = task.Title;
            tbDescription.Text = task.Description;
            cbPriority.SelectedItem = task.Priority;
            cbStatus.SelectedItem = task.Status;
            dpDate.SelectedDate = task.DueDate;
        }
    }
}
```



```

private void Save_Click(object sender, RoutedEventArgs e)
{
    if (string.IsNullOrEmpty(tbTitle.Text) ||
        string.IsNullOrEmpty(tbDescription.Text) ||
        cbPriority.SelectedItem == null ||
        cbStatus.SelectedItem == null ||
        dpDate.SelectedDate == null)
    {
        MessageBox.Show(
            "Заполните все поля.",
            "Ошибка",
            MessageBoxButton.OK,
            MessageBoxImage.Warning);

        return;
    }

    Task.Title = tbTitle.Text;
    Task.Description = tbDescription.Text;
    Task.Priority = (TaskPriority)cbPriority.SelectedItem;
    Task.Status = (TaskProcStatus)cbStatus.SelectedItem;
    Task.DueDate = dpDate.SelectedDate.Value;

    DialogResult = true;
}

private void Cancel_Click(object sender, RoutedEventArgs e)
{
    DialogResult = false;
}
}

```

Создаем пользовательский элемент, который будем использовать в главном окне для статистики по количеству заданий с разными статусами

StatisticsControl.xaml.cs

```

public partial class StatisticsControl : UserControl
{
    public StatisticsControl()
    {
        InitializeComponent();
    }

    public void UpdateStats(List<TaskModel> tasks)
    {
        txtNew.Text =
            $"Новые: {tasks.Count(x => x.Status == TaskProcStatus.New)}";

        txtProgress.Text =
            $"В работе: {tasks.Count(x => x.Status == TaskProcStatus.InProgress)}";

        txtCompleted.Text =
            $"Завершённые: {tasks.Count(x => x.Status == TaskProcStatus.Completed)}";
    }
}

```

Создаем главное окно с таблицей заданий, горизонтальным меню и кнопками для управления заданиями

MainWindow.xaml.cs

```
public partial class MainWindow : Window
{
    private readonly ITaskService _service;

    private readonly IFileService _jsonService =
        new JsonFileService();

    private readonly IFileService _xmlService =
        new XmlFileService();

    public MainWindow()
    {
        InitializeComponent();

        _service = new TaskService(new ApplicationContext());

        cbFilter.ItemsSource =
            Enum.GetValues(typeof(TaskProcStatus));

        RefreshGrid();
    }

    private void RefreshGrid()
    {
        tasksGrid.ItemsSource = null;
        tasksGrid.ItemsSource = _service.GetAllTasks();

        stats.UpdateStats(_service.GetAllTasks());
    }
}
```

```
private void Add_Click(object sender, RoutedEventArgs e)
{
    var window = new TaskModalWindow();

    if (window.ShowDialog() == true)
    {
        if (_service.Add(window.Task))
            RefreshGrid();
        else
            MessageBox.Show("Некорректные данные");
    }
}

private void Edit_Click(object sender, RoutedEventArgs e)
{
    var task = tasksGrid.SelectedItem as TaskModel;

    if (task == null)
        return;

    var copy = new TaskModel
    {
        Id = task.Id,
        Title = task.Title,
        Description = task.Description,
        Priority = task.Priority,
        Status = task.Status,
        DueDate = task.DueDate
    };

    var window = new TaskModalWindow(copy);

    if (window.ShowDialog() == true)
    {
        if (_service.Update(window.Task))
            RefreshGrid();
        else
            MessageBox.Show("Изменение невозможно");
    }
}
```

```

private void Delete_Click(object sender, RoutedEventArgs e)
{
    var task = tasksGrid.SelectedItem as TaskModel;

    if (task == null)
        return;

    if (_service.Delete(task))
        RefreshGrid();
}

private void Search_Click(object sender, RoutedEventArgs e)
{
    tasksGrid.ItemsSource = _service.Search(tbSearch.Text);
}

private void Filter_Click(object sender, RoutedEventArgs e)
{
    if (cbFilter.SelectedItem == null)
        return;

    var status = (TaskProcStatus)cbFilter.SelectedItem;

    tasksGrid.ItemsSource =
        _service.GetTasksByStatus(status);
}

```

```

private void SaveJson_Click(object sender, RoutedEventArgs e)
{
    SaveFileDialog dialog = new SaveFileDialog();

    dialog.Filter = "JSON files (*.json)|*.json";

    if (dialog.ShowDialog() == true)
    {
        _jsonService.Save(
            dialog.FileName,
            _service.GetAllTasks());

        MessageBox.Show("Файл сохранён.");
    }
}

private void LoadJson_Click(object sender, RoutedEventArgs e)
{
    OpenFileDialog dialog = new OpenFileDialog();

    dialog.Filter = "JSON files (*.json)|*.json";

    if (dialog.ShowDialog() == true)
    {
        List<TaskModel> tasks =
            _jsonService.Load(dialog.FileName);

        _service.GetAllTasks().Clear();
        _service.GetAllTasks().AddRange(tasks);

        RefreshGrid();
    }
}

```

```

private void SaveXml_Click(object sender, RoutedEventArgs e)
{
    SaveFileDialog dialog = new SaveFileDialog();

    dialog.Filter = "XML files (*.xml)|*.xml";

    if (dialog.ShowDialog() == true)
    {
        _xmlService.Save(
            dialog.FileName,
            _service.GetAllTasks());

        MessageBox.Show("Файл сохранён.");
    }
}

private void LoadXml_Click(object sender, RoutedEventArgs e)
{
    OpenFileDialog dialog = new OpenFileDialog();

    dialog.Filter = "XML files (*.xml)|*.xml";

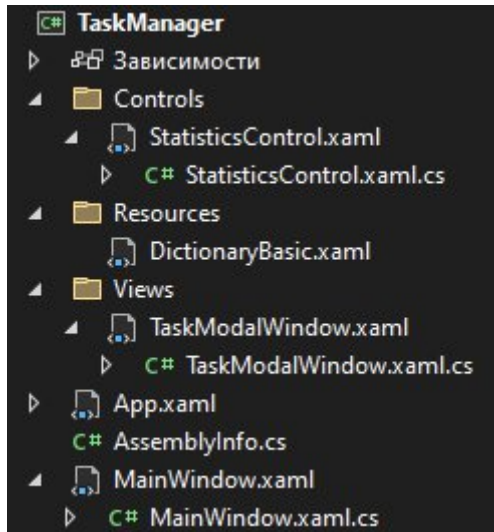
    if (dialog.ShowDialog() == true)
    {
        List<TaskModel> tasks =
            _xmlService.Load(dialog.FileName);

        _service.GetAllTasks().Clear();
        _service.GetAllTasks().AddRange(tasks);

        RefreshGrid();
    }
}

```

Структура проекта



Внешний вид

Task Manager

Файл

Задача

Поиск

Фильтр

Название	Описание	Приоритет	Срок выполнения	Статус
Подготовить отчёт	Отчет по проекту	High	25.06.2026	InProgress
Проверить почту	Ответить на письма	Medium	24.06.2026	New
Обновить документацию	Добавить описание	Low	28.06.2026	Completed
Исправить ошибку	Проверить неверный код	High	26.06.2026	InProgress
Провести тестирование	Проверить работу программы	High	29.06.2026	Completed

Добавить

Изменить

Удалить

Статистика

Новые: 1

В работе: 2

Завершённые: 2

Задача

Название

Описание

Приоритет

Статус

Срок

24.06.2026

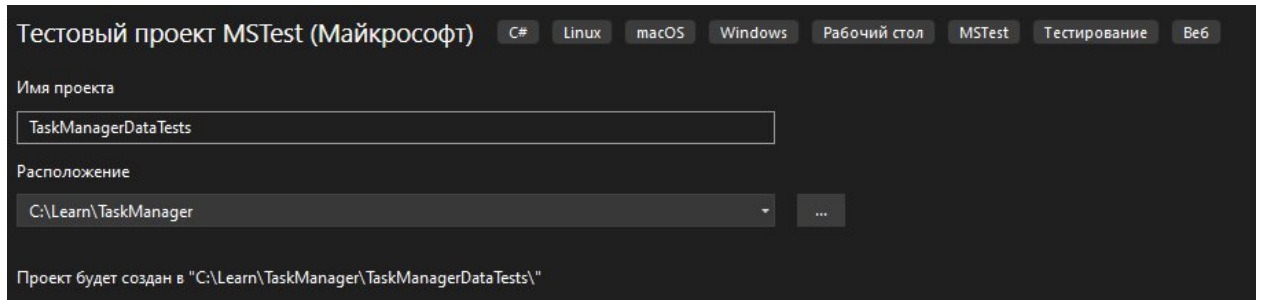
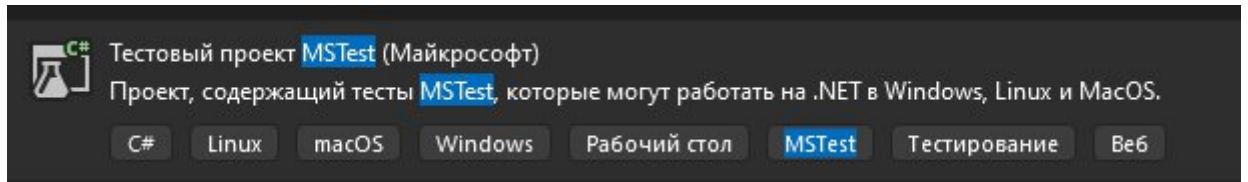
15

Сохранить

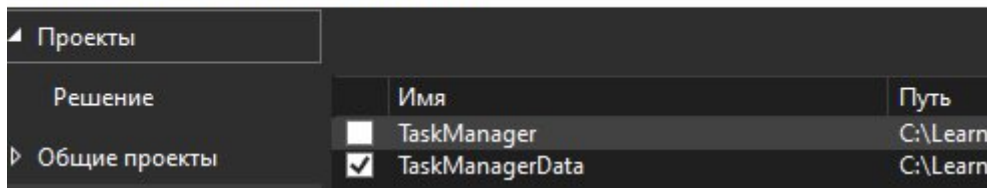
Отмена

Создание модульных тестов для классов-сервисов

Для тестирования классов с поведением программы добавим в TaskManager проект MSTest. И добавим зависимость TaskManagerData для ее тестирования



Диспетчер ссылок - TaskManagerDataTests



Тестирование TaskService

Класс основной логики управления задачами тестируется на инициализацию и успешное выполнение всех реализованных методов: добавление, изменение, удаление задачи, вывод списка задач по статусу, по описанию.

```
[TestClass()]
public class TaskServiceTests
{
    [TestMethod()]
    public void TaskServiceTest()
    {
        ApplicationContext context = new ApplicationContext();
        TaskService svc = new TaskService(context);

        Assert.IsNotNull(svc);
        Assert.IsInstanceOfType(svc, typeof(TaskService));
    }
}
```

```

[TestMethod()]
public void AddTest()
{
    ApplicationContext context = new ApplicationContext();
    TaskService svc = new TaskService(context);

    var task = new TaskModel
    {
        Title = "Задание 1",
        Description = "Описание",
        DueDate = DateTime.Now.AddDays(1),
        Priority = TaskPriority.High
    };

    bool result = svc.Add(task);

    Assert.IsNotNull(svc);
    Assert.IsTrue(result);
    Assert.AreEqual(1, context.Tasks.Count);
}

```

```

[TestMethod()]
public void UpdateTest()
{
    ApplicationContext context = new ApplicationContext();
    TaskService svc = new TaskService(context);
    var task = new TaskModel
    {
        Title = "Задание1",
        DueDate = DateTime.Now.AddDays(1)
    };

    context.Tasks.Add(task);

    var updatedTask = new TaskModel
    {
        Id = task.Id,
        Title = "Задание 1.1",
        DueDate = DateTime.Now.AddDays(2)
    };

    bool result = svc.Update(updatedTask);

    Assert.IsTrue(result);
    Assert.AreEqual("Задание 1.1", context.Tasks[0].Title);
}

```



```

[TestMethod()]
public void DeleteTest()
{
    ApplicationContext context = new ApplicationContext();
    TaskService svc = new TaskService(context);
    var task = new TaskModel
    {
        Title = "Задание 1",
        DueDate = DateTime.Now.AddDays(1)
    };

    context.Tasks.Add(task);

    bool result = svc.Delete(task);

    Assert.IsTrue(result);
    Assert.AreEqual(0, context.Tasks.Count);
}

```

```

[TestMethod()]
public void GetTasksByStatusTest()
{
    ApplicationContext context = new ApplicationContext();
    TaskService svc = new TaskService(context);
    context.Tasks.Add(new TaskModel
    {
        Title = "Написание Отчета",
        DueDate = DateTime.Now.AddDays(1),
        Status = TaskProcStatus.New
    });

    context.Tasks.Add(new TaskModel
    {
        Title = "Тестирование Функций",
        DueDate = DateTime.Now.AddDays(1),
        Status = TaskProcStatus.Completed
    });

    var result = svc.GetTasksByStatus(TaskProcStatus.New);

    Assert.AreEqual(1, result.Count);
    Assert.AreEqual(TaskProcStatus.New, result[0].Status);
}

```

```

[TestMethod()]
public void SearchTest()
{
    ApplicationContext context = new ApplicationContext();
    TaskService svc = new TaskService(context);
    context.Tasks.Add(new TaskModel
    {
        Title = "Написание Отчета",
        Description = "Подробный отчет",
        DueDate = DateTime.Now.AddDays(1)
    });

    var result = svc.Search("отчет");

    Assert.AreEqual(1, result.Count);
}

```

Тестирование FileService

Классы выгрузки/загрузки данные тестируются на методы сохранения данных в файл и загрузки данных в программу из файла.

Для работы с JSON:

```

[TestClass()]
public class JsonFileServiceTests
{
    [TestMethod()]
    public void PathTest()
    {
        JsonFileService svc = new JsonFileService();
        string _path = "test_tasks.json";

        Assert.IsNotNull(svc);
        Assert.IsInstanceOfType(svc, typeof(JsonFileService));
        Assert.IsTrue(File.Exists(_path));
    }
}

```

```
[TestMethod()]
public void SaveTest()
{
    JsonFileService svc = new JsonFileService();
    string _path = "test_tasks.json";

    var tasks = new List<TaskModel>
    {
        new TaskModel
        {
            Title = "Задание 1",
            Description = "Описание",
            DueDate = DateTime.Now.AddDays(1),
            Status = TaskProcStatus.New
        },
        new TaskModel
        {
            Title = "Задание2",
            DueDate = DateTime.Now.AddDays(2)
        }
    };

    svc.Save(_path, tasks);

    Assert.IsTrue(File.Exists(_path));
}
```

```

[TestMethod()]
public void LoadTest()
{
    JsonFileService svc = new JsonFileService();
    string _path = "test_tasks.json";
    var tasks = new List<TaskModel>
    {
        new TaskModel
        {
            Title = "Задание 1",
            Description = "Описание",
            DueDate = DateTime.Now.AddDays(1),
            Status = TaskProcStatus.Completed
        },
        new TaskModel
        {
            Title = "Задание2",
            DueDate = DateTime.Now.AddDays(2)
        }
    };

    svc.Save(_path, tasks);

    var loadedTasks = svc.Load(_path);

    Assert.AreEqual(2, loadedTasks.Count);
    Assert.AreEqual("Задание 1", loadedTasks[0].Title);
    Assert.AreEqual("Задание2", loadedTasks[1].Title);
    Assert.AreEqual(TaskProcStatus.Completed,
        loadedTasks[0].Status);
    Assert.AreEqual(TaskProcStatus.New,
        loadedTasks[1].Status);
}

```

Для работы с XML:


```

[TestClass()]
public class XmlFileServiceTests
{
    [TestMethod()]
    public void PathTest()
    {
        XmlFileService svc = new XmlFileService();
        string _path = "test_tasks.xml";

        Assert.IsNotNull(svc);
        Assert.IsInstanceOfType(svc, typeof(XmlFileService));
        Assert.IsTrue(File.Exists(_path));
    }
}

```

```

[TestMethod()]
public void SaveTest()
{
    XmlFileService svc = new XmlFileService();
    string _path = "test_tasks.xml";

    var tasks = new List<TaskModel>
    {
        new TaskModel
        {
            Title = "Задание 1",
            Description = "Описание",
            DueDate = DateTime.Now.AddDays(1),
            Status = TaskProcStatus.New
        },
        new TaskModel
        {
            Title = "Задание2",
            DueDate = DateTime.Now.AddDays(2)
        }
    };

    svc.Save(_path, tasks);

    Assert.IsTrue(File.Exists(_path));
}

```

```

[TestMethod()]
public void LoadTest()
{
    XmlFileService svc = new XmlFileService();
    string _path = "test_tasks.xml";
    var tasks = new List<TaskModel>
    {
        new TaskModel
        {
            Title = "Задание 1",
            Description = "Описание",
            DueDate = DateTime.Now.AddDays(1),
            Status = TaskProcStatus.Completed
        },
        new TaskModel
        {
            Title = "Задание2",
            DueDate = DateTime.Now.AddDays(2)
        }
    };

    svc.Save(_path, tasks);

    var loadedTasks = svc.Load(_path);

    Assert.AreEqual(2, loadedTasks.Count);
    Assert.AreEqual("Задание 1", loadedTasks[0].Title);
    Assert.AreEqual("Задание2", loadedTasks[1].Title);
    Assert.AreEqual(TaskProcStatus.Completed,
        loadedTasks[0].Status);
    Assert.AreEqual(TaskProcStatus.New,
        loadedTasks[1].Status);
}

```

Результаты тестирования

Все тесты проходят проверку

